Python Break Statement

Last Updated : 5 Jun, 2026

The break statement in Python is used to immediately terminate a for or while loop when a specified condition is met. After the loop exits, program execution continues with the next statement following the loop.

Example: Let's start with an example to understand how break works in a loop.

```
a = [1, 3, 5, 7, 9, 11]
val = 7

for i in a:
    if i == val:
        print(f"Found at {i}!")
        break
else:
    print(f"not found")
```

```
Found at 7!
```

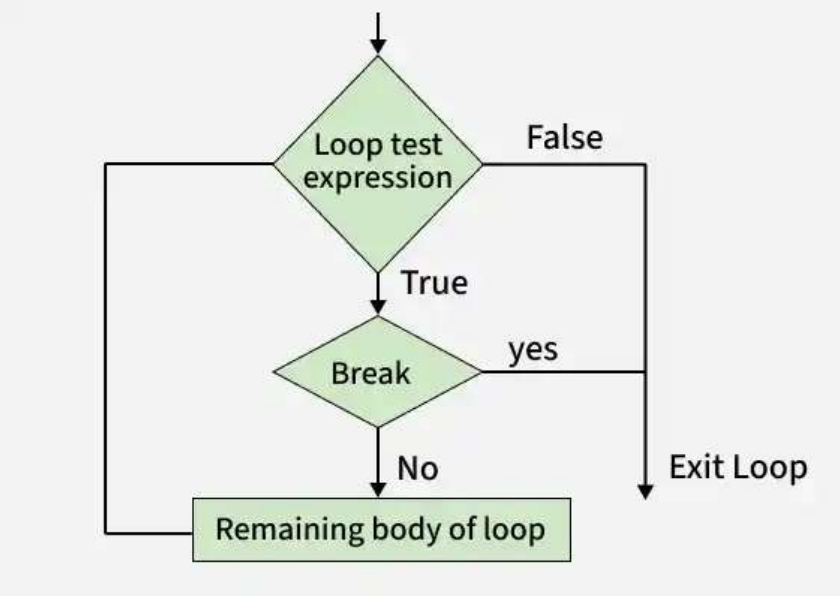
Output

```
Found at 7!
```

Explanation:

- The loop iterates through each number in the list.
- When the number 7 is found, it prints a confirmation message and executes break, exiting the loop immediately.
- If the loop completes without finding the number, the else block is executed.

Let's understand the break statement with a loop using a flowchart:



Break statement Flow

Break Statement with for Loop

A for loop in Python iterates over a sequence (like a list, tuple, string or range) and executes a block of code for each item in that sequence. The break statement can be used within a for loop to exit the loop before it has iterated over all items, based on a specified condition.

```
for i in range(10):
    print(i)
    if i == 6:
        break
```

```
0
1
2
3
4
5
6
```

Output

```
0
1
2
3
4
5
6
```

Break Statement with while Loop

A while loop in Python repeatedly executes a block of code as long as a specified condition is True. The break statement can be used within a while loop to exit the loop based on dynamic conditions that may not be known beforehand.

```
c = 5
```

```
while True:
    print(c)
    c -= 1
    if c == 0:
        print("Countdown finished!")
        break  # Exit the loop
```

```
5
4
3
2
1
Countdown finished!
```

Output

```
5
4
3
2
1
Countdown finished!
```

Using break in Nested Loops

Nested loops are loops within loops, allowing for more complex iterations, such as iterating over multi-dimensional data structures. When using the break statement within nested loops, it's essential to understand its scope:

- **Innermost Loop Exit:** A break statement will only exit the loop in which it is directly placed (the nearest enclosing loop).
- **Exiting Multiple Loops:** To exit multiple levels of nested loops, additional strategies are required, such as using flags or encapsulating loops within functions.

Example: If we have a list of lists (a 2D list) and we want to find a specific number. Once we find the number, we want to stop searching in both the inner and outer loops.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
val = 5
found = False

for r in matrix:
    for n in r:
        if n == val:
            print(f"{val} found!")
            found = True
            break  # Exit the inner loop
    if found:
```

```
        break  # Exit the outer loop

    if found:
```

Output

```
5 found!
```

Explanation:

- Outer loop iterates through each row in the matrix.
- Inner loop checks each number in the current row.
- When the value 5 is found, the message "5 found!" is printed and found variable is set to True and break exits the inner loop immediately.

Loops in Python help automate repetitive tasks, but sometimes you may need to exit a loop, skip an iteration, or alter the normal flow of execution. These actions are handled using loop control statements. Python supports the following control/jump statements:

- continue statement
- break statement
- pass statement

Related Articles

- *Nested loops*
- *while loop in Python*
- *for loop in Python*

Break In Python Visit Course

Comment N nikhila... 81

Explore

- Python Fundamentals
- Python Data Structures
- Advanced Python
- Data Science with Python
- Web Development with Python
- Python Practice
- Courses

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

Python Tutorial | Data Types | Interview Questions | Examples | Quizzes | DSA Python | Data Science | NumPy | Pandas | Practice